

MoneyTracker — Documento Completo

App iOS finanza personale per il mercato italiano · Aggiornato: agosto 2026 · Versione 3.x

Nome visualizzato: l'app compare sotto l'icona come **“Bucktrail”** (INFOPLIST_KEY_CFBundleDisplayName). Il nome tecnico del progetto — cartella, target Xcode, bundle identifier (com.matteo.moneytracker.MoneyTracker) — resta MoneyTracker e non cambia: questo documento e il codice continuano a riferirsi al progetto con quel nome.

1. Visione e Posizionamento

L'idea centrale

Una singola app, pagamento una tantum, che sostituisce carta e penna, fogli Excel, quattro app diverse, il commercialista per le cose semplici, e la testa piena di conti da fare. Apri l'app e sai esattamente dove sei, dove stai andando, e cosa puoi permetterti adesso.

Posizionamento in una frase

“L'unica app italiana che conosce le tue banche, capisce le tue abitudini, e ti dice esattamente cosa puoi permetterti — adesso.”

Differenziatori chiave

- **Italiana di design e cultura:** non una traduzione di un'app americana. Pensata per conti italiani, banche italiane, fisco italiano.
- **Completamente offline:** zero dipendenza da server, zero rischio di shutdown.
- **Una tantum:** in un mercato dominato da abbonamenti, il pagamento unico è un messaggio di fiducia.
- **Privacy radicale:** nessun dato condiviso, nessuna pubblicità, nessun tracking.
- **Design minimale:** funziona anche per chi non ha mai usato un'app di finanza.

Modello di business

Attuale: distribuzione TestFlight con Apple ID personale (certificato 7 giorni).

Al lancio App Store: acquisto una tantum €4,99 — accesso completo, nessun abbonamento, nessuna pubblicità.

Futuro (monetizzazione aggiuntiva): Piano Premium opzionale €1,99/mese per connessione bancaria automatica, AI Financial Coach, sync Supabase cross-device, Modalità Coppia/Famiglia.

2. Stack Tecnico e Architettura

Componente	Tecnologia
Linguaggio	Swift 6.0 (strict concurrency)
UI Framework	SwiftUI
Persistenza	SwiftData (@Model, ModelContext, FetchDescriptor)
Concorrenza	Swift 6 @MainActor, structured concurrency con Task
Notifiche	UserNotifications (UNCalendarNotificationTrigger, UNTimeIntervalNotificationTrigger)
Shortcuts/Siri	AppIntents framework (AppIntent, AppShortcutsProvider)
Logging	OSLog (Logger.persistence, .recurring, .classifier)
Sicurezza DB	NSFileProtectionComplete su SQLite WAL/SHM
Cloud Sync	Supabase (SDK integrato, in sviluppo)
iOS target	iOS 17+
Localizzazione	7 lingue (IT, EN, ES, FR, DE, ZH-Hans, JA)
Architettura	MVVM implicita, computed properties SwiftUI-native
Xcode project	PBXFileSystemSynchronizedRootGroup (auto-scan cartelle)

Pattern architetturali

- **Singleton:** NotificationManager.shared, FormatterCache (entrambi @MainActor)
 - **NSCache:** CategoryClassifier.resultCache per risultati di classificazione
 - **Transferable:** CSVFile con DataRepresentation (nessun file temp su disco)
 - **AppStorage:** persistenza preferenze UI (hideBalance, monthlyReportEnabled, filtri)
 - **UserDefaults:** soglie saldo (keyed by account UUID, evita migration SwiftData)
-

3. Struttura del Progetto

```

MoneyTracker/
├─ MoneyTrackerApp.swift      ← Source principale
├─ seed categorie
├─ Auth/
│   ├── AppLockGate.swift      ← Blocco biometrico app
│   ├── BiometricAuthenticating.swift
│   ├── LoginView.swift        ← Login Supabase
│   ├── SignUpView.swift
│   └─ SupabaseManager.swift   ← Client Supabase, auth, sync
├─ Debug/
│   └─ DemoDataSeeder.swift     ← Dati demo per testing
├─ Domain/
│   ├── AccountBalanceCache.swift ← Cache saldi calcolati
│   ├── AuditLogger.swift
│   ├── CategoryClassifying.swift ← Protocollo classificazione
│   └─ KeywordCategoryClassifier.swift ← Classificatore keyword on-
device
│   ├── NotificationManager.swift ← Notifiche locali budget/saldo
│   ├── NotificationScheduling.swift
│   └─ RecurringTransactionEngine.swift ← Engine ricorrenze
automatiche
├─ Export/
│   └─ PDFReportGenerator.swift ← Generazione PDF mensile
├─ Formatting/
│   ├── CurrencyFormatting.swift ← Estensioni Double/Date
│   └─ FormatterCache.swift      ← Singleton formatter costosi
├─ Intents/
│   └─ AddExpenseIntent.swift    ← Apple Shortcuts integration
├─ Models/
│   ├── Account.swift
│   ├── Budget.swift
│   ├── Category.swift
│   ├── Goal.swift
│   └─ Transaction.swift
├─ OnboardingTour/
│   ├── TabBarFrameCapture.swift
│   ├── ContextualHint.swift
│   ├── TourManager.swift
│   ├── TourOverlayView.swift
│   └─ TourStep.swift
├─ Persistence/
│   ├── Logging.swift
│   ├── ModelContextSaving.swift
│   └─ SchemaMigration.swift
├─ Resources/
│   └─ CategoryKeywords.json     ← Keywords per classificazione
automatica
├─ Sync/
│   └─ SyncService.swift         ← Sync Supabase (in sviluppo)

```

```
├─ Views/
│   ├── AccountsView.swift
│   ├── AddTransactionView.swift
│   ├── AddTransferView.swift
│   ├── AuditLogView.swift
│   ├── BudgetView.swift
│   ├── CategoryManagementView.swift
│   ├── ContentView.swift      ← Tab bar principale
│   ├── DashboardView.swift    ← Home
│   ├── GoalsView.swift
│   ├── PianificaView.swift
│   ├── SettingsView.swift
│   ├── StatisticsView.swift
│   ├── Theme.swift            ← Design system completo
│   └── TransactionsView.swift
├─ [*.lproj]                   ← Localizzazioni
(it/en/de/fr/es/ja/zh-Hans)
├─ Assets.xcassets/
MoneyTrackerTests/             ← Unit test (XCTest)
MoneyTrackerUITests/          ← UI test (XCUITest, simula uso
reale)
├─ UITestSupport.swift         ← Classe base, helper navigazione/FAB
├─ TransactionFlowUITests.swift ← Aggiungi/modifica/elimina movimento
├─ AccountFlowUITests.swift    ← Aggiungi/modifica/elimina conto
├─ CategoryFlowUITests.swift   ← Aggiungi/modifica/elimina categoria
└─ NavigationStressUITests.swift ← Uso intensivo: tab switching,
sheet ripetuti
docs/
├─ MoneyTracker.pdf            ← Questo documento (unico
riferimento)
graphify-out/                  ← Knowledge graph del progetto
```

Automazione notturna (fuori dal repo, in ~/Desktop/NightTestApp/): uno script (run_nightly_tests.sh) lanciato ogni notte alle 4:00 da un LaunchAgent (~/Library/LaunchAgents/com.moneytracker.nightlytests.plist) esegue gli XCUITest su simulatore; se falliscono, invoca Claude Code headless per analizzare e correggere il codice, riprova fino a 5 volte, scrive report_notturmo.md nella root del progetto e apre una Pull Request con le correzioni. Vedi sezione 10 per i dettagli del funzionamento.

4. Modelli Dati

Account

Rappresenta un conto bancario, carta, portafoglio o investimento.

Campo	Tipo	Descrizione
id	UUID	Identificatore univoco

Campo	Tipo	Descrizione
name	String	Nome del conto (es. “Carta Unicredit”)
type	String	Tipo: conto corrente, carta, contanti, investimento
initialBalance	Double	Saldo iniziale inserito dall’utente
isArchived	Bool	Se archiviato, non appare nelle liste attive
isExcludedFromTotal	Bool	Se escluso, non concorre al saldo totale in home
colorHex	String	Colore personalizzato

Computed: currentBalance (solo tx isDone == true), futureBalance (tutte le tx), plannedDelta (differenza futuro–attuale).

Relazioni: transactions: [Transaction] (cascade delete), budgets: [Budget] (cascade delete).

Transaction

Campo	Tipo	Descrizione
id	UUID	Identificatore univoco
name	String	Descrizione (es. “Affitto”, “Stipendio”)
amount	Double	Importo
date	Date	Data della transazione
type	String	“expense” o “income”
isFixed	Bool	Spesa/entrata fissa ricorrente
isDone	Bool	true = già effettuata, false = pianificata
isTransfer	Bool	Fa parte di un trasferimento tra conti
transferGroupId	String	UUID condiviso tra le due gambe di un trasferimento
category	String	Categoria assegnata
recurringFrequency	String	Frequenza: vuoto / daily / weekly / monthly / yearly
account	Account?	Relazione con il conto

Budget

Campo	Tipo	Descrizione
id	UUID	Identificatore univoco

Campo	Tipo	Descrizione
category	String	Categoria a cui si applica
limit	Double	Limite mensile in valuta
account	Account?	Opzionale: limita il budget a un conto specifico

Goal (Obiettivo di risparmio)

Campo	Tipo	Descrizione
id	UUID	Identificatore univoco
name	String	Nome obiettivo
emoji	String	Emoji selezionata dall'utente
targetAmount	Double	Importo target
currentAmount	Double	Importo risparmiato finora
hasDeadline	Bool	Se ha una scadenza
deadline	Date?	Data scadenza opzionale

Category

Campo	Tipo	Descrizione
id	UUID	Identificatore univoco
name	String	Nome categoria
iconName	String	Nome SF Symbol
isDefault	Bool	Le categorie default non sono eliminabili

FormatterCache (performance)

Singleton centralizzato per formatter costosi. Ricrea i formatter solo quando cambia la valuta o la locale — elimina N allocazioni NumberFormatter/DateFormatter per ogni render di ogni row.

- `currencyFormatter()` — NumberFormatter currency riutilizzato
- `monthYear` — DateFormatter("MMMM yyyy") singleton
- `dayMonth` — DateFormatter("d MMM") singleton
- `shortMonth` — DateFormatter("MMM") singleton per grafici

5. Design System (Theme.swift)

Palette colori semantica

Token	Descrizione
DS.in	Testo principale (nero in light, bianco in dark)

Token	Descrizione
DS.paper	Sfondo principale
DS.fog	Sfondo secondario (grigio chiarissimo)
DS.smoke	Testo secondario, label, placeholder

Spacing

DS.Space.xs / s / m / l / xl / xxl — sistema a scala fissa per coerenza visiva.

Componenti riutilizzabili

Componente	Descrizione
HeroAmount(amount:size:hidden:)	Importo in grande. Con hidden == true mostra “•••”
SectionLabel(text:)	Intestazione sezione in uppercase con letterspacing
PrimaryButton(title:action:)	Bottone principale nero/bianco
GhostButton(title:action:)	Bottone secondario testo-only
ThinDivider()	Separatore sottile in DS.fog
DSTransactionRow(transaction:)	Riga transazione standard con VoiceOver completo
MonthBar	Barra grafico mensile con label via FormatterCache
EmptyStateView(icon:title:subtitle:cta:)	Empty state coerente con CTA opzionale

Modalità chiara e scura

Full support Light/Dark mode nativo SwiftUI, senza override manuali.

App Icon (Assets.xcassets/AppIcon.appiconset)

Grafico a barre crescenti in stile minimale (DS.ink su light, DS.paper-inverso su dark), con l’ultima barra colorata in oro/ambra come accento — il verde/rosso di DS.positive/DS.negative resta riservato ai saldi in-app, quindi l’icona usa un colore distinto. Tre varianti richieste da iOS 18+: AppIcon_light_1024.png, AppIcon_dark_1024.png, AppIcon_tinted_1024.png (quest’ultima in scala di grigi, colorata dal sistema quando l’utente sceglie un tema icone personalizzato). Tutti e tre i PNG sono 1024×1024 senza canale alpha (richiesto da Apple).

6. Funzionalità Implementate

Home (DashboardView)

- **Saldo Attuale Hero** — totale conti inclusi in grande (HeroAmount, ~56pt bold)
- **Chips conti** — riga orizzontale scrollabile con nome + saldo. Pulsante ☰ apre pannello selezione/ordinamento (HomeAccountOrderSheet). Ordine persistito via @AppStorage
- **Nascondi saldo** — pulsante occhio: HeroAmount e chips mostrano “•••”. Stato persistito via @AppStorage("hideBalance")
- **Saldo Atteso** — appare solo se ci sono movimenti pianificati. Mostra saldo futuro + delta (▲/▼)
- **Fissi del mese** — mini-card affiancate “Spese fisse”/“Entrate fisse”: somma di tutte le transazioni isFixed del mese corrente (pagate o da pagare, isDone ignorato). Ciascuna mostrabile/nascondibile indipendentemente da Impostazioni → Home (@AppStorage("showFixedExpensesTotal")/("showFixedIncomeTotal")); rispetta “Nascondi saldo” (hideBalance)
- **Ultimi Movimenti** — collassabile. Ultime 4 transazioni non fisse. Swipe destra = Modifica, sinistra = Elimina
- **Toolbar:** pulsante tema (sinistra) + occhio + ingranaggio (destra)

Movimenti (TransactionsView)

- Lista completa per data (più recenti in cima)
- **Filtri a due righe:** Tutti · Fisse · Trasf. · Uscite · Entrate + tutte le categorie. Filtri combinabili
- **Filtri avanzati** persistiti via @AppStorage (importo min/max, date)
- Statistiche mensili in header con cambio mese (freccie avanti/indietro)
- **Swipe destra:** “Fatto/Pianificato” per spese fisse e trasferimenti; “Modifica” per le altre
- **Swipe sinistra:** Elimina (singola o entrambe le gambe del trasferimento via transferGroupId)

Aggiunta Transazione (AddTransactionView)

- Campi: nome (con categorizzazione automatica on-device), importo, tipo (uscita/entrata), categoria, conto, data
- Toggle “Già effettuato” (isDone), Toggle “Spesa fissa” (isFixed)
- Attivando “Spesa fissa” disattiva automaticamente “Già effettuato”
- Categorizzazione automatica: keyword matching on-device senza rete. L’utente può sovrascrivere manualmente
- Keyboard navigation: importo → nome → .done (auto-save)
- Reset form corretto al riutilizzo (load()) resetta tutto quando editing == nil)

Trasferimenti (AddTransferView)

- Seleziona conto “Da” e “A”, inserisci importo
- Toggle “Già effettuato” (default: attivo)
- Genera due transazioni collegate con stesso transferGroupId
- Bottone dinamico: “Trasferisci” o “Pianifica” in base allo stato

Conti (AccountsView)

- Lista: prima conti inclusi nel totale, poi esclusi (badge “escluso”)
- **Swipe:** destra = Modifica, sinistra = Elimina (con haptic)
- **Context menu** (long press): Modifica / Escludi-Includi / Archivia / Elimina
- **Conti archiviati:** sezione separata, opacità 35%. Context menu: Ripristina / Elimina definitivamente
- Hero: saldo attuale + futuro dei conti inclusi
- Nuovo conto non escluso → aggiunto automaticamente in home
- **Modifica saldo attuale:** il campo mostra e accetta sempre il saldo *reale* (`currentBalance`), non il saldo iniziale interno. Su un conto con transazioni già registrate, digitare un nuovo valore non sovrascrive lo storico: viene creata una transazione di rettifica (“Rettifica saldo”, categoria dedicata) per la sola differenza, datata oggi — così il conto riparte esattamente dal numero inserito senza dover registrare le spese/entrate mancanti una per una. Su un conto senza transazioni il valore aggiorna direttamente il saldo iniziale, come prima.

Budget (BudgetView)

- Budget per categoria con limite mensile in valuta
- Barra di avanzamento (verde/rosso per soglia superata)
- Filtro opzionale per conto specifico
- Storico budget (ultimi mesi) via `BudgetHistorySheet`
- Notifiche locali quando ci si avvicina o supera il limite

Obiettivi (GoalsView)

- Nome + emoji, importo target, importo attuale, deadline opzionale
- Barra di avanzamento e percentuale completamento
- VoiceOver completo: nome, %, importi, deadline, stato completato

Statistiche (StatisticsView)

- Grafico a torta per categoria (solo tx `isDone == true`)
- Trend mensile a barre (entrate vs uscite ultimi 6 mesi)
- Confronto annuale: mese corrente vs stesso mese anno scorso
- Esportazione PDF mensile (via `PDFReportGenerator`)
- Esportazione CSV (via `Transferable`, nessun file temp su disco)
- Loading indicator animato durante generazione PDF/CSV

Impostazioni (SettingsView)

- Cambio valuta (€, \$, £, ¥, ecc.) — si aggiorna dinamicamente ovunque via `Double.currencySymbol`
- Cambio lingua (7 lingue)
- Selezione tema
- Toggle “Report mensile PDF” — notifica locale il 1° di ogni mese alle 10:00
- Avviso saldo basso per conto (persistito in `UserDefaults`)
- Sezione “Home”: toggle indipendenti per mostrare/nascondere le mini-card “Spese fisse mensili”/“Entrate fisse mensili”

Biometria (Auth/)

- AppLockGate: blocco app con Face ID / Touch ID via LocalAuthentication
- Login/SignUp Supabase (in sviluppo per sync cloud)

Shortcuts Apple (Intents/)

- Azione “Aggiungi spesa” disponibile nell’app Comandi di iOS
- Aggiungibile come widget su schermata home o di blocco
- Inserisci nome e importo → salvato direttamente nel database senza aprire l’app
- try? safe init del ModelContainer (no crash in caso di store non disponibile)

Onboarding Tour (OnboardingTour/)

Due livelli: - **Livello 1 — panoramica** (TourStep/OverviewStep): sequenza lineare mostrata una volta sola al primo avvio, uno step per tab con spotlight sulla sola icona della tab bar. - **Livello 2 — mini-tour contestuali** (ContextualHint): un singolo spotlight + card con “Ho capito”, mostrato una volta sola la prima volta che l’utente arriva davvero su un elemento (dopo aver completato la panoramica).

Altri componenti: - TourManager: stato pubblicato per entrambi i livelli, persistenza (UserDefaults) di “panoramica completata” e “hint visti” - TourOverlayView: overlay unico (ModalCard, StepCard, HintCard) che disegna sia la panoramica sia gli hint contestuali — montato in MoneyTrackerApp quando isActive **oppure** activeHint != nil - TabBarFrameCapture: cattura frame tab bar per posizionamento overlay

Ricorrenze Automatiche

- Campo “Ricorrenza” in AddTransactionView: nessuna / settimanale / mensile / annuale
- Transaction.recurringFrequency: String nel modello;
Transaction.templateId: String collega ogni copia generata alla transazione “template” originale (vuoto sul template stesso)
- RecurringTransactionActor.processRecurring() genera automaticamente, per il solo periodo corrente, le occorrenze mancanti dei template attivi — mai per tutti i mesi futuri in blocco, per non sfasare il saldo
- Le nuove tx vengono create come isDone = false (pianificate), stesso nome/importo/categoria/conto del template, data corrispondente nel nuovo periodo
- **Trigger:** gira su un @ModelActor in background sia al lancio dell’app sia a ogni ritorno in foreground (scenePhase == .active, MoneyTrackerApp.swift) — prima girava solo al lancio da zero del processo, quindi restava silente per chi teneva l’app viva in background attraverso il cambio di mese. Una guardia giornaliera (UserDefaults) evita elaborazioni ridondanti.
- **Interrompi ricorrenza:** voce nel context menu (long press) di TransactionsView, disponibile sia sul template originale sia su una qualunque copia già generata (risale al template via templateId) — svuota recurringFrequency sul template senza toccare le occorrenze già create.

Localizzazione

7 lingue complete (IT, EN, ES, FR, DE, ZH-Hans, JA). 340 chiavi per file. Accessibility labels localizzate con `String(localized:)` (non string interpolation italiana).

Haptic Feedback

- Swipe “Modifica”: haptic soft
- Swipe “Elimina”: haptic medium
- Tutti i bottoni distruttivi: haptic medium

7. Audit Qualità (Score: 7.2 / 10)

Area	Voto
Architettura	7/10
Qualità codice / Bug	7.5/10
Performance	7/10
Standards / Manutenibilità	7.5/10
Sicurezza	8/10
Totale	7.2/10

Cosa funziona bene

- Swift 6 strict concurrency rispettata ovunque — zero data races
- OSLog strutturato con subsystem/category separati
- `NSFileProtectionComplete` sul database `SwiftData`
- Formula injection prevention nel CSV export
- Design system coerente (DS namespace, `EmptyStateView`, `HeroAmount`)
- `LazyVStack` per liste lunghe (no List con altezza fissa)
- Notifiche con ID idempotenti (remove-then-add)
- VoiceOver completo su tutti i componenti principali

Bug risolti (sessioni precedenti)

ID	Problema	Stato
BUG-01	DashboardView List + frame fisso → testo troncato con Dynamic Type	✅ Fixato → LazyVStack
BUG-02	GoalsView.load() → “1000.0” invece di “1000”	✅ Fixato → truncatingRemainder
BUG-03	CategoryStat.id UUID instabile → rompeva diffing SwiftUI	✅ Fixato → computed var stabile su name
BUG-04	print() di debug in TabBarFrameCapture → console noise su layout hot-path	✅ Fixato → rimossi 7 print

ID	Problema	Stato
PERF-01	Allocazione NumberFormatter/DateFormatter per ogni render	✅ Fixato → FormatterCache singleton
SEC-01	try! su ModelContainer in Shortcuts → crash dell'estensione	✅ Fixato → try? + guard con errore user-visible
i18n-01	Sezione Siri non tradotta in nessuna lingua	✅ Fixato → 10 chiavi aggiunte a tutti i 6 file
i18n-02	Accessibility labels in italiano hardcoded	✅ Fixato → String(localized:) ovunque
PERF-02	€ hardcoded in AddAccountView e AddTransferView	✅ Fixato → Double.currencySymbol

Issue residue (non critiche)

ID	Problema	Priorità
PERF-R01	Account.currentBalance O(n_tx) — scala male con >5000 tx	Alta — v2
PERF-R02	@Query senza fetchLimit in DashboardView	Media
SMELL-01	CategoryManagementView sezione vuota → bare Text	Bassa
SMELL-02	AddGoalView, AddBudgetView senza @FocusState	Media
SEC-02	NotificationManager.isEnabled non verifica autorizzazione OS effettiva	Media
SEC-04	CheckBalanceIntent biometric policy rimossa (API iOS 26 cambiata)	Bassa

8. Roadmap

Livello 2 — Necessario per essere competitivi

Feature	Stato
Tour onboarding / discoverability	⚙ In sviluppo (OnboardingTour/ presente)

Feature	Stato
Widget schermata Home e Lock Screen (WidgetKit + AppGroup)	☐
Ricorrenze automatiche	☒ Implementato
Apple Watch App (watchOS target + WatchConnectivity)	☐
Face ID / Touch ID blocco app	☒ Implementato (AppLockGate)

Livello 3 — Feature WOW (differenziazione totale)

Feature	Descrizione
Connessione bancaria automatica	GoCardless/Nordigen API (gratuito EU) — Intesa, UniCredit, Fineco, BancoBPM, Poste
“Me lo posso permettere?”	Shortcut conversazionale on-device: “scarpe 150€” → risposta immediata con contesto budget
AI Financial Coach	Report mensile automatico il 1° del mese, on-device. Opzionale: sintesi via Claude Haiku API
Tracker abbonamenti	Rilevamento automatico pagamenti ricorrenti, alert per abbonamenti non usati
Punteggio Salute Finanziaria	Score 0–100 settimanale su 5 dimensioni (budget, obiettivi, fondo emergenza, spese fisse, risparmio)
Sync cross-device (Supabase)	iPhone + iPad, offline-first, TLS + AES-256, last-write-wins
Modalità Coppia/Famiglia	Database condiviso via CloudKit o Supabase
Split spese	Divisione conto con contatti iPhone + iMessage/WhatsApp automatico
Calendario flusso di cassa	Vista calendario con proiezione saldo giorno per giorno
Tracciatore Patrimonio Netto	Attivi – Passivi (banca + investimenti + immobili – mutuo + debiti)
Feature fiscali italiane	Bollette tracker, F24 tracker, Detrazioni fiscali 19%, 730 helper, TFR tracker
Modalità “Stipendio → Zero”	Zero-based budgeting guidato (metodologia YNAB)

Livello 4 — Futuro

Investimenti automatici (Directa/Fineco API), Crypto wallet tracker, FIRE Calculator, “Boomerang” confronto periodo anno scorso, Spesa emotiva, Marketplace partner opt-in, API pubblica.

9. Setup e Installazione

Requisiti

- Mac con macOS 13+
- Xcode (gratuito dall'App Store del Mac, ~7 GB)
- iPhone con iOS 17+
- Cavo USB (prima volta)

Installazione da sorgente

1. Apri MoneyTracker.xcodeproj in Xcode
2. Signing & Capabilities → seleziona il tuo Team (Apple ID)
3. Collega iPhone al Mac con cavo → “Autorizza” sul telefono
4. Seleziona il tuo iPhone come dispositivo target
5. Premi ► (Play)
6. Prima volta: Impostazioni iPhone → Privacy e sicurezza → App per sviluppatori → Autorizza il tuo Apple ID

Configurare il banner Shortcuts

1. App Comandi sul iPhone → “+” → “Aggiungi azione” → cerca “Money Tracker”
2. Seleziona “Aggiungi spesa”
3. . . . → “Aggiungi a schermata Home” o “Aggiungi a schermata di blocco”
4. Tocca l'icona → banner in cima → inserisci nome e importo → salvato senza aprire l'app

Note

- **Certificato:** con Apple ID gratuito scade ogni 7 giorni. Ricollegare il telefono e premere Play per rinnovarlo.
- **Dati al sicuro:** SwiftData non cancella i dati durante gli aggiornamenti.
- **Trasferimento:** richiede almeno 2 conti non archiviati.

Errori comuni

Problema	Soluzione
“Signing failed”	Xcode → Signing & Capabilities → Team → seleziona Apple ID
App non si apre	Impostazioni → Privacy → App per sviluppatori → Autorizza
Certificato scaduto (7 giorni)	Ricollega iPhone, premi Play in Xcode
“Personal teams do not support iCloud”	Non aggiungere iCloud/Push con Apple ID gratuito

10. Changelog

v3.x (settembre 2026) — Tre bug critici: Tocco posteriore che “scompare”, perdita dati al login, ricorrenze arretrate

Tre segnalazioni indipendenti dell'utente, tutte con causa radice nella sincronizzazione locale↔cloud e nel motore ricorrenze.

Bug fix — la spesa aggiunta da Tocco posteriore/Siri/Shortcuts non compariva mai nell'app: - **Bug:** `AddExpenseIntent/AddIncomeIntent` (`Intents/AddExpenseIntent.swift`) girano fuori-processo rispetto all'app (`openAppWhenRun = false`, necessario perché il Tocco posteriore non deve aprire l'app). Senza un App Group configurato, quel processo separato e l'app principale aprono due `ModelConfiguration` senza url: esplicito, che risolvono in due container sandbox **diversi** — l'entitlements file era completamente vuoto, nessun App Group presente. Il salvataggio riusciva davvero (da qui il banner “salvato!”), ma finiva in uno store che l'app non apre mai. - **Fix:** nuovo `Persistence/SharedStore.swift` — App Group `group.com.matteo.moneytracker.shared` aggiunto a `MoneyTracker.entitlements`; sia `MoneyTrackerApp.init()` sia `IntentContainer` in `AddExpenseIntent.swift` ora aprono lo store tramite lo stesso url: nel container condiviso. Migrazione automatica una tantum (`migrateLegacyStoreIfNeeded`) che copia (non sposta) lo store dalla vecchia posizione a quella condivisa al primo lancio dopo l'attivazione — altrimenti il nuovo percorso risulterebbe vuoto e l'utente perderebbe tutto. Se la capability non è (ancora) attiva in Xcode, `mainStoreURL` è nil e si ripiega sul comportamento precedente: nessuna regressione, ma il bug resta finché il passo manuale in Xcode (Signing & Capabilities → App Groups) non viene fatto.

Bug fix — apertura app con tutti i dati cancellati, sostituiti da uno snapshot vecchio di mesi: - **Bug:** `MoneyTrackerApp.swift` chiamava `SyncService.clearLocalData()` in automatico ogni volta che `auth.isLoggedIn` diventava false — ma quel cambio di stato scatta anche per una sessione persa **passivamente** (token di refresh scaduto/fallito, rete instabile: il SDK Supabase emette comunque un evento “signed out”), non solo per un logout intenzionale. Il risultato: al primo hiccup di rete l'app svuotava silenziosamente tutto lo store locale, poi al login successivo lo ripopolava con l'ultimo snapshot su Supabase — spesso vecchio di settimane/mesi se il push locale era rimasto indietro (es. proprio a causa del bug precedente sul Tocco posteriore). - **Fix:** rimosso lo svuotamento automatico reattivo. Il logout esplicito (bottone “Esci” in `SettingsView`) ora chiama `clearLocalData()` da solo, dopo aver tentato un push delle modifiche pendenti — stesso pattern già usato per l'eliminazione account. L'isolamento multi-utente resta garantito da `syncOnLogin()`, che svuota il locale solo quando rileva davvero un login con un `userId` diverso dall'ultimo salvato. - **Indurimento aggiuntivo** in `SyncService.pull()`: le transazioni locali ancora “sporche” (create/modificate ma non ancora confermate su Supabase — es. `markTransactionDirty` chiamato dall'Intent ma il push non ancora partito) non vengono più cancellate solo perché assenti dall'ultimo pull; prima venivano trattate come “cancellate su un altro dispositivo”.

Bug fix — le transazioni ricorrenti non comparivano automaticamente all'inizio del mese: - **Bug:** il fix precedente (v3.x agosto 2026, “fix trigger ricorrenze”) faceva ripartire `processRecurring()` ad ogni ritorno in foreground, ma generava solo l'occorrenza del periodo **corrente** — se l'app non veniva aperta esattamente a cavallo del cambio mese/settimana/anno, quell'occorrenza restava persa per sempre, non recuperata nei lanci successivi. - **Fix:** `RecurringTransactionActor.processRecurring()` ora recupera tutte le occorrenze mancanti dall'ultima generata fino a oggi (backfill), non solo quella

corrente — fino a un tetto di sicurezza di 60 per template (5 anni per le mensili) per evitare che un template dimenticato a lungo generi migliaia di righe in un colpo solo; oltre il tetto, il recupero prosegue nei run successivi. La ricerca dell'ultima occorrenza generata usa ora un fetch mirato per `templateId` invece di una finestra fissa di 2 mesi, che perdeva di vista le ricorrenze annuali e causava un duplicato ad ogni run. Coperto da `MoneyTrackerTests/RecurringTransactionEngineTests.swift` (backfill di mesi arretrati, non-duplicazione delle annuali dopo 2+ mesi, template appena creato).

Correttezza — precisione degli importi migrati da Double a Decimal

(SchemaMigration.swift): - **Bug:** la migrazione V1→V2 usava `Decimal(Double)`, che congela l'espansione binaria imprecisa del `Double` (es. 19.99 diventava 19.9899999999999488 per sempre). - **Fix:** conversione tramite la rappresentazione testuale più breve del `Double` (`Decimal(string: String(value))`), che restituisce l'importo "pulito" effettivamente inserito dall'utente. Le closure di migrazione ora propagano gli errori invece di inghiottirli con `try?`, coerente con il fallback già esistente in `MoneyTrackerApp.init()` in caso di store non migrabile.

⚠ **Passo manuale richiesto per il fix del Tocco posteriore:** in Xcode, target "MoneyTracker" → Signing & Capabilities → "+ Capability" → "App Groups" → verificare/aggiungere `group.com.matteo.moneytracker.shared`. Finché non viene fatto, il Tocco posteriore continua a comportarsi come prima (fix inattivo, nessuna regressione).

v3.x (agosto 2026) — Nuova icona, nuovo nome "Bucktrail", grafico trend a barre affiancate

Identità visiva — nuova App Icon: - Sostituito il vecchio grafico bianco/nero (barre + linea spezzata con pallini) con un design più pulito: barre crescenti minimali con l'ultima colorata in oro/ambra come unico accento cromatico. - Aggiunte le varianti `AppIcon_dark_1024.png` e `AppIcon_tinted_1024.png` (prima dichiarate in `Contents.json` ma mai popolate — l'icona non cambiava mai in dark mode o con temi icona personalizzati iOS 18+).

Nome visualizzato → "Bucktrail": - Impostato `INFOPLIST_KEY_CFBundleDisplayName = Bucktrail` su entrambe le configurazioni Debug/Release del target MoneyTracker in `project.pbxproj`. Il nome tecnico del progetto (cartella, target, bundle id) non cambia — vedi nota in testa a questo documento.

Statistiche — trend mensile da barre+linea a barre affiancate: -

`StatisticsView.swift`: il grafico "Uscite" (barre) + "Entrate" (linea con pallini) è stato sostituito con due `BarMark` affiancate per mese (`.position(by:)`), una per entrate e una per uscite — più leggibile del confronto barra/linea, specialmente quando le entrate erano ricorrenti a importo simile. - Rimosso il parametro `isDot` di `legendItem(color:label:)`, diventato morto dato che entrambe le serie sono ora barre (nessuna legenda "a pallino" residua).

v3.x (agosto 2026) — Rettifica saldo conto, fix trigger ricorrenze, interrompi ricorrenza

Bug fix — modifica del saldo di un conto con storico produceva un risultato sbagliato: - **Bug:** il campo "Saldo attuale" in `AddAccountView` mostrava e sovrascriveva `Account.initialBalance`, che è solo l'ancora sommata a *tutte* le transazioni storiche

(AccountBalanceCache). Su un conto con transazioni già registrate, scrivere un nuovo valore non produceva mai quel saldo: il risultato finale restava nuovo_valore + storico, non il numero digitato. - **Fix:** il campo ora precompila e accetta il saldo *reale* (currentBalance). In salvataggio, se il conto ha già transazioni, viene creata una transazione di rettifica (“Rettifica saldo”, nuova categoria di default) per la sola differenza, datata oggi, invece di toccare initialBalance — lo storico resta intatto e il saldo risultante coincide esattamente col valore inserito. Su un conto senza transazioni il comportamento resta identico a prima (aggiorna direttamente initialBalance). - Nuova categoria di default “Rettifica saldo” (Category.swift), aggiunta anche retroattivamente ai profili già esistenti tramite la migrazione già presente in seedIfNeeded.

Bug fix — le transazioni ricorrenti non venivano riportate al mese nuovo: - **Bug:** RecurringTransactionActor.processRecurring() (logica di generazione corretta e già presente) veniva invocato solo dentro un .task legato al primo montaggio di ContentView, cioè solo a un lancio “da zero” dell’app. Chi teneva l’app viva in background attraverso il cambio di mese non vedeva mai generare le nuove occorrenze finché non chiudeva e riapriva il processo. - **Fix:** processRecurring() viene ora invocato anche a ogni ritorno in foreground (scenePhase == .active, MoneyTrackerApp.swift). La guardia giornaliera già esistente evita elaborazioni ridondanti nello stesso giorno.

Nuova funzionalità — “Interrompi ricorrenza”: - Nuova voce nel context menu (long press) di TransactionsView, disponibile sia sulla transazione template originale sia su una qualunque copia già generata (risale al template tramite Transaction.templateId) — così non serve risalire a una transazione creata magari mesi o anni fa. Svuota recurringFrequency sul template: nessuna transazione già esistente viene toccata, semplicemente non ne verranno generate di nuove nei mesi successivi.

i18n: aggiunte le chiavi “Rettifica saldo” e “Interrompi ricorrenza” a tutti i 6 file di traduzione non italiani (EN/FR/ES/DE/JA/ZH-Hans).

v3.x (luglio 2026) — Card “Fissi del mese” in Home + tour onboarding testato end-to-end

Nuova funzionalità — totali spese/entrate fisse del mese in Home: - Nuova sezione “Fissi del mese” in DashboardView (tra “Saldo atteso” e “Ultimi movimenti”): due mini-card affiancate (FixedTotalTile, nuovo componente in Theme.swift) con il totale di tutte le transazioni isFixed del mese corrente, separate per Uscita/Entrata — a prescindere da isDone, così l’utente vede il totale dei fissi (affitto, auto, assicurazione, ecc.) indipendentemente dal fatto che siano già stati pagati. - Ciascuna delle due card è mostrabile/nascondibile **indipendentemente** dall’altra da Impostazioni → nuova sezione “Home” (due Toggle separati), persistiti via @AppStorage("showFixedExpensesTotal")/("showFixedIncomeTotal"), default entrambi attivi. - Rispetta “Nascondi saldo” (hideBalance): con l’occhio disattivato mostra “•••” come il resto della Home. - Testi tradotti nelle 6 lingue non italiane (riuso della chiave esistente “Fissi del mese” per il titolo sezione, nuove chiavi “Spese fisse”/“Entrate fisse” per le card e “Spese fisse mensili”/“Entrate fisse mensili” per i toggle in Impostazioni).

QA — tour di onboarding verificato end-to-end prima del checkpoint: - Aggiunti accessibilityIdentifier (tourModalCTA, tourNext, tourBack, tourHintGotIt) ai bottoni di TourOverlayView.swift: il solo testo “Avanti” era ambiguo negli XCUITest perché iOS assegna automaticamente la stessa etichetta di accessibilità (in

italiano) alla chevron “mese successivo” di MonthBar, un componente estraneo al tour presente su più tab. - Nuovo MoneyTrackerUITests/TourFlowUITests.swift: percorre l'intera panoramica (17 step, incluse le due card modali iniziale/finale) con screenshot ad ogni passaggio, verificando titolo e presenza del testo atteso per ciascuno step. Utile come test di regressione per future modifiche al tour. - Verificato manualmente, screenshot per screenshot, che ogni spotlight punti all'elemento corretto e che i cambi tab/segmento (Movimenti → Statistiche → Pianifica → Conti) avvengano senza artefatti; nessun problema riscontrato.

v3.x (luglio 2026) — Trasferimenti: fix eliminazione e nuova modalità di modifica

Bug fix — eliminazione trasferimento non funzionava dentro la sezione “Trasf.” (TransactionsView.swift): - **Bug:** il dialog di conferma “Elimina entrambi i movimenti” era agganciato solo alla vista hub (root della NavigationStack), non alla vista della sezione realmente in primo piano quando si è dentro “Trasferimenti”. Un .confirmationDialog/.alert attaccato a una vista non in cima allo stack di navigazione resta “in sospeso”: compariva solo tornando indietro all'hub, mai subito dopo lo swipe. - **Fix:** il modificatore (estratto in View.transferDeleteConfirmation(pending: onConfirm:)) è ora agganciato sia all'hub sia a subListContent(for:) (la vista pushata per ciascuna sezione), così l'eliminazione funziona subito indipendentemente da dove viene avviata.

Nuova funzionalità — modifica di un trasferimento esistente: - Prima, toccare un trasferimento mostrava solo un alert bloccante (“elimina e ricrea”): non era possibile modificarlo. - AddTransferView ora accetta un parametro opzionale editing: Transaction? (una delle due gambe collegate). In modalità modifica, save() aggiorna **entrambe** le gambe esistenti in modo sincronizzato (importo, data, note, stato fatto/pianificato, conti “Da”/“A”) invece di crearne di nuove — il ruolo (.uscita/.entrata) e il transferGroupId di ciascuna gamba non vengono mai alterati, solo il conto assegnato. - TransactionsView: il tap su un trasferimento apre ora AddTransferView(editing:) in una sheet, al posto dell'alert bloccante rimosso. - Aggiunta la traduzione di “Modifica trasferimento” nelle 6 lingue non italiane.

v3.x (luglio 2026) — Colori semantici positivo/negativo, fix ricorrenze, tour a due livelli riscritto

Colori (verde/rosso), indipendenti dal tema attivo: - Nuovi DS.positive/DS.negative in Theme.swift (toni smorzati, adattivi chiaro/scuro) + DS.signColor(_) come unica fonte di verità per “colora in base al segno” - Saldi (Home, Conti) colorati per segno; uscite/entrate colorate ovunque appaiano come riga (Movimenti, Home, storico ricorrenze) tranne i trasferimenti (restano neutri: non sono un guadagno/perdita reale) - Statistiche: “Uscite”/“Entrate” e relativi grafici (barre, linea, legenda, barre per categoria), confronto mese su mese e anno su anno colorati in base a miglioramento/peggioramento - Budget: resta neutro entro il limite, rosso solo se sfiorato - Rimosso un flag morto (FixedExpenseRow.colorCoded) mai attivato da nessuna parte che avrebbe dovuto occuparsi di questo

Bug fix — motore ricorrenze automatiche (RecurringTransactionEngine.swift): - **Bug:** quando una transazione con recurringFrequency impostata (Settimanale/Mensile/Annuale) veniva creata, il motore che genera le occorrenze future non considerava che il template stesso copre già il proprio periodo

(mensile/settimanale/annuale) — perché il template è escluso dal fetch delle transazioni “normali” (`recurringFrequency.isEmpty`). Risultato: al lancio successivo dell’app veniva generata una copia duplicata per lo stesso mese/settimana/anno del template. - **Fix:** aggiunto un controllo esplicito che salta la generazione se la data del template ricade già nel periodo corrente. - Verificato separatamente che il rilevamento passivo delle ricorrenze (`RecurringSeriesDetector`, tab “Ricorrenti”) **non** esclude le transazioni da Tutti/Uscite/Entrate/Trasf. — sono due sistemi distinti, il secondo è solo di lettura/analisi.

Tour onboarding — riscritto da “un’icona per tab” a step sui singoli elementi:

Livello 1 (panoramica) ora ha 16 step-elemento (Home ×4, Movimenti ×3, Statistiche ×3, Pianifica ×3, Conti ×1) invece di un semplice spotlight sull’icona della tab bar; motore di spotlight anchor-based condiviso col Livello 2 - Aggiunti campi di tuning per-step (`spotlightPadding`, `spotlightYOffset`, `secondaryAnchorID`, `cardAtTop`) per correggere ritagli troppo piccoli/grandi, margini asimmetrici e card che coprivano il contenuto spiegato - Lo step “Tutti i movimenti” ora naviga davvero dentro la sezione e ne mostra il contenuto reale mentre lo spiega, richiudendola quando si avanza - Fine tour: torna sempre alla Home (prima restava sull’ultima tab visitata) - Rimossa la spiegazione del FAB “+” ovunque (sia il mini-tour post-tour sia lo step dedicato in Conti) e i mini-tour per barra di ricerca/spese ricorrenti — giudicati poco chiari - Bug fix: bottone “Elimina” nello swipe (Movimenti) ereditava un tint ambientale pensato per altro, mostrando sfondo bianco/icona nera in tema scuro invece del rosso di sistema - Bug fix: la tastiera della ricerca in Movimenti ricompariva da sola tornando all’hub dopo aver aperto una sezione — il focus non veniva mai resettato

v3.x (luglio 2026) — Fix mini-tour contestuali che non comparivano mai

- **Bug:** `MoneyTrackerApp` montava `TourOverlayView` solo quando `tourManager.isActive` era vero (panoramica in corso). I mini-tour contestuali di Livello 2 (`ContextualHint`, il tooltip “Ho capito” tab per tab) scattano invece **dopo** che la panoramica è finita — la view che li disegna non esisteva più in quel momento, quindi non comparivano mai.
- **Fix:** la condizione di montaggio ora è `tourManager.isActive || tourManager.activeHint != nil` (`MoneyTrackerApp.swift`).
- **Bug correlato:** al termine della panoramica l’utente resta sull’ultima tab visitata durante il tour; il trigger dei mini-tour in `ContentView` scattava solo su `onChange(of: selectedTab)`, quindi l’hint di quella tab non partiva mai finché non se ne usciva e vi si tornava. Aggiunto un secondo trigger su `onChange(of: tourManager.isActive)` che controlla subito la tab corrente a fine tour.
- **Contenuto:** aggiunto il mini-tour hint `goalsList` (anchor `goalsList`, già presente in `PianificaView` ma orfano) per spiegare gli obiettivi di risparmio quando si passa al segmento “Obiettivi” — prima quella sezione non aveva alcuna spiegazione contestuale. Rifiniti alcuni testi italiani per un tono più professionale (`hint.tabStatistiche.body`, `hint.movimentiHub.body`, `hint.fab.body`).

v3.x (luglio 2026) — Fix tocco posteriore/Siri: transazioni che sparivano dopo il salvataggio

- **Bug:** `AddExpenseIntent/AddIncomeIntent` (App Intent usati dal tocco posteriore e da Siri/Comandi Rapidi) girano in un processo separato dall’app principale. Il salvataggio locale andava a buon fine (da qui il messaggio “salvato!”), ma il loro `ctx.save()` non generava la notifica `ModelContext.didSave` osservata da `SyncService.recordChanges()` nel processo principale — quindi la transazione non veniva mai marcata come “da sincronizzare”. Alla riapertura dell’app, la

sequenza push→pull di SyncService trattava la transazione (mai arrivata su Supabase) come cancellata da un altro dispositivo e la eliminava anche in locale.

- **Fix:** aggiunto `SyncService.markTransactionDirty(_:)`, chiamato esplicitamente da entrambi gli Intent subito dopo `ctx.save()`, per marcare la transazione come dirty a prescindere dal processo in cui è stata creata (vedi `Sync/SyncService.swift`, `Intents/AddExpenseIntent.swift`).

v3.x (luglio 2026) — XCUITest e automazione notturna

- Nuovo target **MoneyTrackerUITests** (XCUITest) nel progetto Xcode, creato via script Ruby (`xcodeproj gem`) sul `project.pbxproj`
- 12 test UI in 4 classi: flusso movimenti, conti, categorie, e uno stress test di navigazione (tab switching rapido, apertura/chiusura ripetuta di fogli, aggiunta/eliminazione multipla)
- Aggiunto hook `MoneyTrackerApp.isUITesting` (`--uitesting` come launch argument): bypassa login Supabase, Face ID e tour di onboarding, forza la modalità demo con dati deterministici — nessuna chiamata di rete durante i test
- Aggiunti `accessibilityIdentifier` su tutti gli elementi interattivi chiave (campi, bottoni, swipe actions) in `AddTransactionView`, `AccountsView`, `CategoryManagementView`, `SettingsView`, `TransactionsView`, `ContentView (FAB)`, `BudgetView`, `GoalsView`
- `TestPlan.xctestplan` aggiornato per includere il nuovo target
- **Routine notturna:** script `run_nightly_tests.sh` + `LaunchAgent` `com.moneytracker.nightlytests.plist` (fuori dal repo, in `~/Desktop/NightTestApp/`) — esegue i test ogni notte alle 4:00, corregge automaticamente il codice con Claude Code headless in caso di fallimento (fino a 5 tentativi), scrive `report_notturmo.md` e apre una PR

v3.x (luglio 2026) — Riorganizzazione e documentazione

- Riorganizzata struttura cartella: `Views/`, `Domain/`, `Models/`, `Auth/`, `Export/`, `Intents/`, `Debug/`, `Persistence/`, `Formatting/`, `OnboardingTour/`, `Sync/`
- Eliminati file residui (`.Rhistory`, `worktree` stantio)
- Tutta la documentazione consolidata in questo unico file

v3 (giugno–luglio 2026) — Enterprise audit e bug fix

43 miglioramenti su 6 aree:

- **FormatterCache:** eliminata allocazione `NumberFormatter/DateFormatter` per ogni render (hot path)
- **DashboardView:** List fissa → `LazyVStack` (Dynamic Type fix)
- **GoalsView:** `EmptyStateView`, VoiceOver, bug `load()` con “1000.0”
- **BudgetView:** `EmptyStateView`, VoiceOver su `BudgetRow`
- **AddTransactionView:** keyboard navigation con `@FocusState`
- **AddTransferView:** `.submitLabel(.done)` su importo
- **AddExpenseIntent:** `try!` → `try? safe init` (SEC-01 critico)
- **StatisticsView:** `CategoryStat.id` stabile, `shortMonthFormatted` via `FormatterCache`
- **TransactionsView:** `filteredSnapshot` per accesso centralizzato
- **NotificationManager:** alert saldo basso per singolo conto
- **SettingsView:** loading indicator CSV/PDF, toggle report mensile, animazioni spring

- **i18n**: sezione Siri localizzata in 6 lingue, accessibility labels con `String(localized:)`
- **QA**: rimossi 7 `print()` di debug da `TabBarFrameCapture` e `TourOverlayView`

v2 (primavera 2026) — Feature complete

- Conti multipli con saldo attuale / saldo atteso
- Trasferimenti tra conti (inclusi pianificati)
- Budget per categoria con filtro per conto
- Obiettivi di risparmio
- Shortcut Apple (banner senza aprire l'app)
- Multi-lingua 7 lingue
- Statistiche mensili con grafici
- Esportazione CSV e PDF
- Ricorrenze automatiche (`recurringFrequency`)
- Biometria (Face ID / Touch ID)
- Onboarding tour (in sviluppo)

v1 (inverno 2025) — Base

- Progetto iniziale: transazioni, categorie, saldo singolo conto

11. Workflow Checkpoint

Ogni volta che si fa un checkpoint significativo:

1. **Aggiorna questo documento** (`docs/MoneyTracker.md`) con le modifiche apportate
2. **Rigenera il PDF**: `cd /Users/matteo/Desktop/MoneyTracker && pandoc docs/MoneyTracker.md -o docs/MoneyTracker.pdf --pdf-engine=weasyprint`
3. **Commit e push**: `git add -A && git commit -m "checkpoint: [descrizione]" && git push`
4. **Aggiorna il grafo**: `/graphify --update` in Claude per aggiornare `graphify-out/`

Documento unico — aggiornato ad ogni checkpoint · Generato con Claude (Anthropic)